Guide to APIs

A reference for website and app developers — how APIs are built, secured, and organized, plus a categorized inventory of what's available to build with

---

Contents

• Part I — Architecture & Design Paradigms

• Part II — Authentication & Authorization

• Part III — Rate Limiting & Throttling

• Part IV — Versioning

• Part V — Pagination Patterns

• Part VI — Response Design

• Part VII — Documentation & Discoverability

• Part VIII — Error Handling Conventions

• Part IX — Common Failure Modes

• Part X — APIs by Category (Reference Inventory)

Part I — Architecture & Design Paradigms

REST (Representational State Transfer)

The dominant standard. Resource-based (nouns, not verbs): **GET /users/123**, **POST /orders**. Stateless — each request carries everything needed, no server-side session. Uses HTTP verbs semantically (GET/POST/PUT/PATCH/DELETE) and HTTP status codes (200/201/400/401/403/404/429/500) to convey meaning. Not a strict spec — “RESTful” is a set of conventions, loosely enforced in practice.

GraphQL

Query language, single endpoint (**/graphql**) instead of many resource endpoints. Client specifies exactly which fields it wants — solves REST's over-fetching (getting a full object when you needed one field) and under-fetching (needing multiple round-trips to assemble related data). Trade-off: harder to cache (POST-based, no URL-per-resource), and a single malformed nested query can be far more expensive server-side than a REST call — requires query depth/complexity limiting.

SOAP

XML-based, older enterprise standard. Strict contract (WSDL), heavier payloads, built-in error handling and transaction support. Still common in banking, healthcare (HL7/FHIR uses some SOAP legacy), government/insurance systems — anywhere with entrenched legacy infra and heavy compliance requirements. Mostly displaced by REST/GraphQL for new builds.

gRPC

Binary protocol (Protocol Buffers) over HTTP/2. Much faster and smaller payload than JSON-based REST — used for internal service-to-service communication (microservices talking to each other) rather than public-facing APIs, since it's not human-readable and requires generated client code per language.

Webhooks

Inverse of a normal API call — instead of polling for changes, the service pushes an HTTP POST to your endpoint when an event happens (a payment succeeds, a build completes, a job finishes). Requires you to run a receiving endpoint. Always verify the signature header (HMAC) to confirm the payload actually came from the claimed source, not a spoofed request — this is the single most common webhook security gap.

Part II — Authentication & Authorization

- **API key** — simplest, a static string in a header or query param. Weakest security (no expiration by default, easy to leak in logs/URLs) but ubiquitous for low-stakes APIs.
- **Bearer token** — key sent in an Authorization: Bearer header rather than the URL — marginally safer (doesn't end up in server access logs the way a query-string key does).
- **OAuth 2.0** — the standard for “let this app act on my behalf on another service.” Involves a redirect flow, an access token (short-lived) and refresh token (long-lived, used to get new access tokens without re-prompting the user). Scopes limit exactly what the token can do (read-only vs. write, specific resource types).
- **JWT (JSON Web Token)** — a signed, self-contained token carrying claims (user ID, expiration, roles) that the server can verify without a database lookup — common for stateless auth in modern web apps.
- **HMAC request signing** — used by AWS and others: instead of sending a static secret, you sign each request with a hash of the request contents plus a secret key, so a leaked/intercepted request can't be replayed as-is.
- **mTLS (mutual TLS)** — both client and server present certificates — high-security enterprise/financial API pattern.

Part III — Rate Limiting & Throttling

Typically expressed per-window: requests/minute, requests/day. Communicated via response headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset — not universal, varies by provider). **429 Too Many Requests** is the standard status code, often paired with a Retry-After header telling you how long to wait.

Client-side handling strategies: exponential backoff with jitter (wait progressively longer between retries, with randomness added so many clients don't retry in lockstep), token bucket/leaky bucket algorithms for smoothing your own outbound request rate, request queuing. Some APIs tier by cost, not just count — GraphQL query complexity scoring, or per-endpoint weighting (a search endpoint might “cost” 5x a simple GET).

Part IV — Versioning

- **URL-based** — /v1/users, /v2/users — most common, most visible, easiest for clients to reason about
- **Header-based** — Accept: application/vnd.api+json;version=2 — cleaner URLs, less discoverable
- **Query param** — ?version=2 — rare, usually a sign of a less mature API
- **No versioning** (continuous deployment with backward-compat guarantees) — Stripe's approach: dated versions (2024-06-20) pinned per API key, old behavior preserved indefinitely unless the client opts into a newer date

Part V — Pagination Patterns

- **Offset/limit** — ?offset=40&limit=20 — simple, but breaks if records are inserted/deleted between page requests (page drift)
- **Cursor-based** — ?after=<opaque_token> — stable under concurrent writes, doesn't allow jumping to an arbitrary page number
- **Page-number based** — ?page=3 — user-friendly, same drift risk as offset

Part VI — Response Design

- **Envelope vs. bare** — some APIs wrap responses ({"data": {...}, "meta": {...}}), others return the resource directly. Envelopes make error/pagination metadata consistent but add a layer of unwrapping for every consumer.

• **Partial responses / field selection** (?fields=id,name) — REST's answer to GraphQL's over-fetching problem, offered by some mature REST APIs.

• **HATEOAS** (Hypermedia as the Engine of Application State) — theoretically “pure” REST includes links to related actions/resources in every response so a client can navigate without hardcoded URLs. Rarely implemented in practice — most “REST” APIs are really just resource-oriented HTTP+JSON, not true HATEOAS.

Part VII — Documentation & Discoverability

• **OpenAPI/Swagger spec** — machine-readable API description (endpoints, params, schemas) that generates docs, client SDKs, and mock servers automatically. The dominant standard for REST.

• **Postman collections** — shareable pre-built request sets, often provided alongside docs for quick testing.

• **MCP (Model Context Protocol)** — a newer, different category: not a way to describe a REST API, but a standard for exposing tools/capabilities directly to an LLM agent — the model discovers available tools and their schemas at runtime rather than a human reading docs.

Part VIII — Error Handling Conventions

HTTP status code carries the category (4xx = client error, 5xx = server error), but the useful detail is almost always in the JSON body — error code, human message, sometimes a request_id for support correlation.

Idempotency keys — a client-generated unique ID attached to a request (usually for POST operations like payments) so that if the request is retried after a timeout, the server can recognize it as a duplicate rather than double-processing. This is the single biggest silent gap in webhook/payment integrations — most naive implementations don't build this in from the start.

Part IX — Common Failure Modes Worth Knowing

• **N+1 problem** — calling a list endpoint, then calling a detail endpoint once per item instead of using a batch/expand parameter — devastating at scale, the single most common inefficiency in API integration code.

• **Clock skew** — signed requests (AWS-style HMAC, JWT expiration) failing because the client's system clock has drifted — a classic “works locally, fails in prod” bug.

• **Silent breaking changes** — an API provider changes a field type or removes a field without bumping the version — why pinning to a dated/versioned API and monitoring changelogs matters for anything production-facing.

• **CORS** — browser-only issue (server-to-server calls are unaffected) — the API server must explicitly allow the calling origin, or browser-based JS calls get blocked regardless of whether the API itself would have accepted the request.

Part X — APIs by Category (Reference Inventory)

A broad inventory of API providers by domain, useful as a starting map when scoping what a project needs. Government, scientific, and standards-body sources are almost uniformly free and structurally rich; consumer-platform APIs (social, real estate) have been steadily closing off free access in favor of paid/partner-only tiers; most “enterprise” categories gate the same underlying public-ish data behind aggregation and normalization rather than genuinely proprietary collection.

Government / Public Data

Federal: Regulations.gov, Congress.gov, Federal Register, SEC EDGAR, Census Bureau, BLS, FRED, NOAA/NWS, USPTO, openFDA, CDC, EPA (Envirofacts, TRI), Data.gov, USAspending.gov, SAM.gov, GovInfo, NARA, FEC, USDA/FoodData Central, BEA, Treasury fiscal data, FCC (spectrum, broadband maps, complaints), OSHA, NHTSA, CPSC. State/local: LegiScan, individual state SOS business registries, state court e-filing portals, Socrata-based city/state open-data portals (NYC, Chicago, etc.). International: EU Open Data Portal, UK data.gov.uk, World Bank, UN Comtrade, OECD, IMF, WHO Global Health Observatory, Eurostat, Statistics Canada.

Financial / Market Data

Market data: Alpha Vantage, Polygon.io, IEX Cloud, Finnhub, Yahoo Finance (unofficial), Bloomberg/Refinitiv, Quandl/Nasdaq Data Link, Tiingo, Twelve Data, Intrinio. Crypto: CoinGecko, CoinMarketCap, Binance, Coinbase, Etherscan, Glassnode, The Graph. Banking/fintech: Plaid, Stripe, MX, Yodlee, Finicity, Dwolla, Currencylayer/exchangerate-api. Insurance: Verisk (enterprise-tier).

Search / Web

Google (Search, Maps, Places), Bing Search API, Brave Search API, DuckDuckGo instant-answer API, Kagi Search API. SERP/scraping-as-a-service: SerpApi, DataForSEO, ScraperAPI, Bright Data, Zenrows. Archive: Internet Archive/Wayback Machine API, Common Crawl.

Social / Platform

X/Twitter API, Reddit API, Meta/Facebook Graph API, LinkedIn API, YouTube Data API, TikTok API, Discord API, Telegram Bot API, Mastodon (open, federated), Bluesky/AT Protocol (open by design), Pinterest API, Snapchat Marketing API.

Communication

Email: SendGrid, Mailgun, Postmark, Gmail API, Amazon SES, Resend. SMS/voice: Twilio, Vonage, MessageBird, Plivo. Push notifications: Firebase Cloud Messaging, OneSignal, Pusher. Chat/messaging infra: Sendbird, Stream, Twilio Conversations.

AI / ML

Model providers: Anthropic, OpenAI, Google (Gemini), Cohere, Mistral, Hugging Face, Replicate, Together AI, Groq, Perplexity API. Specialized: ElevenLabs (voice), Stability AI (image), Runway (video), AssemblyAI/Deepgram (speech-to-text), Pinecone/Weaviate/Qdrant (vector databases for RAG).

Geospatial / Location

Google Maps/Places, Mapbox, OpenStreetMap (Overpass API), Census geocoding, Here Technologies, What3Words, Esri/ArcGIS, TomTom.

Business / Company Data

Clearbit, Crunchbase, ZoomInfo, OpenCorporates, D&B, Apollo.io, PitchBook (enterprise), People Data Labs, Hunter.io.

News / Media

NewsAPI, GDELT, The Guardian Open Platform, NYT API, AP News API, Mediastack, Currents API, NewsData.io.

Legal

CourtListener/RECAP, Justia, Casetext, Fastcase, Free Law Project tools (Juriscraper), state court open-data portals where they exist.

E-commerce / Retail

Amazon Product Advertising API, Shopify API, eBay API, Keepa, WooCommerce API, BigCommerce API, Etsy API, Walmart Open API.

Weather / Environment

NOAA/NWS, OpenWeatherMap, Weatherstack, Tomorrow.io, AirNow (EPA air quality), PurpleAir (crowdsourced air quality), Climate.gov datasets, USGS (earthquakes, water data, geological survey).

Transportation

FlightAware, FlightRadar24, AviationStack, GTFS transit feeds (Transitland aggregates many agencies), MarineTraffic, Uber/Lyft business APIs, Here/TomTom traffic APIs, NWS marine forecasts.

Health

openFDA, CDC data APIs (incl. WONDER, Social Vulnerability Index), ClinicalTrials.gov, PubMed/NCBI E-utilities, NIH RePORTER, Healthcare.gov marketplace data, CMS Medicare/Medicaid claims and provider data, UMLS, RxNorm/DailyMed, SNOMED CT.

Dev / Infra Utility

GitHub API, GitLab API, npm registry, PyPI/crates.io registries, Cloudflare API, Stripe, Auth0/Okta/Clerk, Vercel/Netlify/Railway, AWS/GCP/Azure, Datadog/Sentry.

Education

College Scorecard (Dept. of Education), IPEDS, Khan Academy API, Open Library/Internet Archive, CommonLit, Google Classroom API.

Real Estate / Property

Zillow (limited/mostly deprecated public API), Rentspree, ATTOM Data, county assessor portals (no national standard), Estated, Realtor.com (unofficial/partner-only).

Sports

ESPN (unofficial but widely used), SportRadar, TheSportsDB (free), Football-Data.org, MySportsFeeds.

Entertainment / Media Metadata

TMDB (free), OMDb, Spotify Web API, Discogs, IGDB, Open Library.

Science / Research

NASA (imagery, exoplanet data, APOD), arXiv API, Semantic Scholar, CrossRef, ORCID, Protein Data Bank, GBIF, USGS Earthquake API.

Energy

EIA (prices, production, consumption — free and extensive), grid operators/ISOs (CAISO, PJM, ERCOT each expose real-time grid data).

Telecom / Connectivity

FCC Broadband Map API, Ookla Speedtest data, numverify (phone validation), Twilio Lookup (carrier/line-type data).

Identity / Verification

Onfido, Jumio, Persona, ID.me, Veriff — KYC/identity verification layer, distinct from data-broker aggregators used for background/investigative lookups.